TABLEAU · LESSON 3 OF 6

Calculations in Tableau: Calculated Fields, Table Calculations and LOD Expressions

Write calculated fields (row-level and aggregate), use string, date, number and logical functions, handle nulls, then master table calculations (running total, percent of total, difference and growth, rank, moving average, and Compute Using) and Level of Detail expressions (FIXED, INCLUDE, EXCLUDE), with every result checked against pandas.

✦ Tableau Desktop / Tableau Public 2026 | Intermediate | ⏱ 150 min

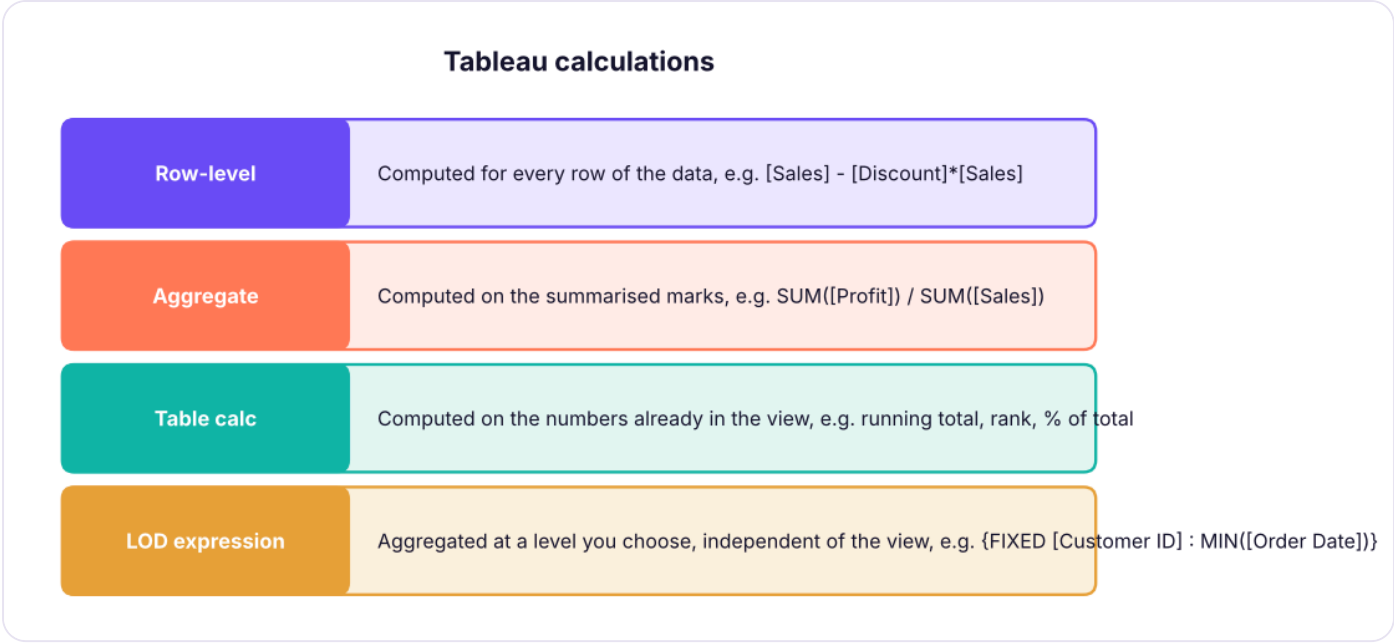
By the end of this lesson you can

- Write row-level and aggregate calculated fields and know which one a question needs
- Use IF/ELSEIF, CASE, IIF, ZN, IFNULL, DATEDIFF, DATETRUNC and string functions
- Apply quick table calculations and control them with Compute Using
- Write FIXED, INCLUDE and EXCLUDE LOD expressions and explain where they sit in the order of operations
- Solve classic interview problems: profit margin, customer first-purchase cohorts and percent of total

Datasets used

`superstore_sales.csv`

Three kinds of calculation



Where each kind of calculation is computed.

Create one with **Analysis → Create Calculated Field** (or right-click a field → Create → Calculated Field). Field names go in square brackets; comments start with `//`.

Python

```
import pandas as pd, numpy as np
df = pd.read_csv("superstore_sales.csv", parse_dates=["order_date", "ship_date"])
print(df[["sales", "profit", "discount"]].describe().round(1).loc[["mean", "min", "max"]])
```

OUTPUT

```
          sales    profit  discount
mean    31854.1    2902.4       0.1
min        42.0  -75873.8       0.0
max    619290.0  149424.9       0.4
```

Row-level vs aggregate: the profit-margin trap

The most common beginner bug in Tableau (and Power BI) is averaging ratios.

Calculated field	Formula	What it means
Margin (row)	`[Profit] / [Sales]`	Margin of each order line; in the view it becomes SUM or AVG of those ratios: **wrong**
Margin (aggregate)	`SUM([Profit]) / SUM([Sales])`	Total profit ÷ total sales for whatever the mark represents: **right**

Python

```
by_cat = df.groupby("category").apply(lambda g: pd.Series({
    "aggregate margin %": g.profit.sum() / g.sales.sum() * 100,
    "average of row margins %": (g.profit / g.sales).mean() * 100}), include_groups=False)
by_cat.round(2)
```

OUTPUT

	aggregate margin %	average of row margins %
category		
Furniture	-2.35	-2.38
Office Supplies	14.95	13.76
Technology	11.29	9.95

3 rows × 2 columns

The two columns differ because the average of row ratios gives a ₹500 order the same weight as a ₹5 lakh order. Format the aggregate field as a percentage (right-click → Default Properties → Number Format → Percentage).

Useful functions

Purpose	Tableau	Example
Logic	`IF … THEN … ELSEIF … ELSE … END`	`IF SUM([Profit]) < 0 THEN "Loss" ELSE "Profit" END`
Logic	`CASE [Region] WHEN "South" THEN … END`, `IIF(test, then, else)`	Short mappings
Nulls	`ZN([Profit])`, `IFNULL([x], 0)`, `ISNULL([x])`	Replace nulls with 0
Dates	`DATEDIFF('day', [Order Date], [Ship Date])`, `DATETRUNC('month', [Order Date])`, `DATEPART('weekday', …)`, `DATEADD`, `TODAY()`	Shipping days, month start
Strings	`LEFT`, `RIGHT`, `MID`, `SPLIT([Customer Name], " ", 1)`, `CONTAINS`, `UPPER`, `TRIM`, `REPLACE`	First name
Numbers	`ROUND`, `ABS`, `CEILING`, `FLOOR`, `INT`	Rounding
Type	`STR()`, `INT()`, `FLOAT()`, `DATE()`	Conversions
Aggregates	`SUM`, `AVG`, `COUNT`, `COUNTD`, `MIN`, `MAX`, `MEDIAN`, `ATTR`	`COUNTD([Order ID])` = orders

Python

```python
df["ship_days"] = (df.ship_date - df.order_date).dt.days          # DATEDIFF('day', [Order Date], [Ship Date])
df["first_name"] = df.customer_name.str.split(" ").str[0]         # SPLIT([Customer Name], " ", 1)
df["order_size"] = np.select([df.sales ⩾ 50000, df.sales ⩾ 10000], ["Large", "Medium"], "Small")   # IF/ELSEIF
print(df.groupby("ship_mode").ship_days.mean().round(1).sort_values())
print(df.order_size.value_counts().to_dict())
```

OUTPUT

```
ship_mode
Same Day          0.0
First Class       1.5
Second Class      3.0
Standard Class    5.6
Name: ship_days, dtype: float64
{'Small': 3359, 'Medium': 1838, 'Large': 1126}
```

Table calculations

A table calculation works on the **numbers already in the view**, so its result depends on the layout. Right-click a measure → **Quick Table Calculation**:

Quick table calc	Use
Running Total	Cumulative sales through the year
Difference / Percent Difference	Month-on-month or year-on-year change
Percent of Total	Share of each category
Rank	Top products
Percentile	Position in a distribution
Moving Average	Smooth noisy monthly data (Edit → previous 2 values + current)
YTD Total / YTD Growth / YoY Growth	Financial reporting

Compute Using (addressing and partitioning)

Compute Using tells Tableau the direction of the calculation. Fields it computes *along* are **addressing** fields; the remaining fields **partition** the view into separate calculations. Example: Category on Rows, MONTH on Columns, Running Total of Sales:

- Compute Using **Table (across)** → running total along months, restarting for each category (partitioned by Category).
- Compute Using **Table (down)** → running total down the categories within each month.

Python

```
m = df[df.order_date.dt.year == 2025].assign(month=lambda x:
x.order_date.dt.to_period("M"))
t = m.pivot_table(index="category", columns="month", values="sales", aggfunc="sum") / 1e5
running = t.cumsum(axis=1)              # Table (across): restarts per category
print(running.iloc[:, [0, 1, 2, -1]].round(1))
share = t.sum(axis=1) / t.values.sum() * 100   # Percent of Total, Table (down)
print((share).round(1).to_dict())
```

OUTPUT

```
month            2025-01  2025-02  2025-03  2025-12
category
Furniture           12.9     16.8     28.7    127.6
Office Supplies      2.6      4.8      9.4     39.7
Technology          38.0     70.8    119.4    616.5
{'Furniture': 16.3, 'Office Supplies': 5.1, 'Technology': 78.7}
```

Common table-calculation functions you write yourself: `RUNNING_SUM(SUM([Sales]))`, `WINDOW_AVG(SUM([Sales]), -2, 0)` (3-month moving average), `LOOKUP(SUM([Sales]), -1)` (previous value), `RANK(SUM([Sales]))`, `INDEX()`, `SIZE()`, `TOTAL(SUM([Sales]))`.

YoY growth as a calculation: `(SUM([Sales]) - LOOKUP(SUM([Sales]), -1)) / ABS(LOOKUP(SUM([Sales]), -1))` with YEAR(Order Date) in the view.

Python

```
y = df.groupby(df.order_date.dt.year).sales.sum()
print(((y / y.shift(1) - 1) * 100).round(1).dropna().to_dict(), "% YoY")
```

OUTPUT

```
{2024: 13.5, 2025: 19.8} % YoY
```

Level of Detail (LOD) expressions

LODs compute an aggregate at a level **you** choose, regardless of the dimensions in the view.

Syntax	Meaning	Example
`{FIXED [Dim] : AGG([Measure])}`	Exactly at [Dim], ignoring the view (but after context filters)	`{FIXED [Customer ID] : MIN([Order Date])}` first purchase
`{INCLUDE [Dim] : AGG(...)}`	View's dimensions **plus** [Dim]	Average sales **per customer** by region
`{EXCLUDE [Dim] : AGG(...)}`	View's dimensions **minus** [Dim]	Region total next to each category, for % of region
`{ : SUM([Sales])}`	Table-scoped (grand total)	% of grand total without table calcs

FIXED: customer cohorts by first-purchase year

`[First Purchase] = {FIXED [Customer ID] : MIN([Order Date])}`, then `YEAR([First Purchase])` on Rows and `COUNTD([Customer ID])` gives new customers per year: an acquisition cohort.

Python

```
first = df.groupby("customer_id").order_date.min().rename("first_purchase")
d2 = df.merge(first, on="customer_id")
cohort = d2.groupby([d2.first_purchase.dt.year.rename("cohort"),
d2.order_date.dt.year.rename("order year")]).customer_id.nunique().unstack(fill_value=0)
cohort
```

OUTPUT

order year	2023	2024	2025
cohort			
2023	540	397	415
2024	0	193	156
2025	0	0	52

3 rows × 3 columns

Read it row by row: customers acquired in 2023 (first row) and how many of them bought again in 2024 and 2025. This retention table is a favourite interview task.

INCLUDE: average sales per customer by region

`AVG({INCLUDE [Customer ID] : SUM([Sales])})` with Region in the view first sums sales per customer *within* each region, then averages those customer totals.

Python

```python
per_cust = df.groupby(["region", "customer_id"]).sales.sum().groupby("region").mean() / 1e3
print((per_cust).round(1).rename("avg sales per customer (₹ thousand)"))
```

OUTPUT

```
region
East     238.4
North    263.5
South    261.9
West     262.4
Name: avg sales per customer (₹ thousand), dtype: float64
```

EXCLUDE: share of the region

With Region and Category in the view, `SUM([Sales]) / SUM({EXCLUDE [Category] : SUM([Sales])})` gives each category's share of its region.

Python

```python
rc = df.groupby(["region", "category"]).sales.sum()
(rc / rc.groupby(level=0).transform("sum") * 100).round(1).unstack()
```

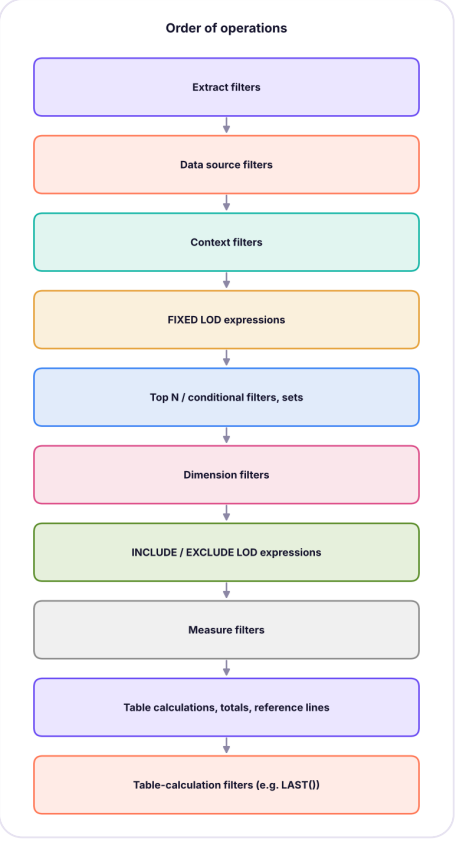
OUTPUT

category	Furniture	Office Supplies	Technology
region			
East	17.1	5.9	77.1
North	16.3	4.5	79.2
South	17.8	4.6	77.6
West	18.0	5.2	76.8

4 rows × 3 columns

Order of operations

Filters and calculations run in a fixed order. This explains most "why did my number not change?" questions:



Tableau's order of operations (simplified). FIXED LODs ignore normal dimension filters unless the filter is added to context.

TIP **Make a FIXED LOD respect a filter**

A FIXED LOD is computed before dimension filters, so filtering Region does not change `{FIXED [Customer ID] : MIN([Order Date])}`. To make it respect the filter, right-click the filter → **Add to Context**.

Summary

KEY TAKEAWAYS

- Row-level calculations run per row; aggregate calculations run on marks: compute margins as SUM/SUM.
- Table calculations work on the view's numbers; Compute Using sets the direction (addressing) and the restart points (partitioning).
- FIXED sets an exact level; INCLUDE adds a level; EXCLUDE removes one.
- FIXED LODs run before dimension filters; use context filters when they must respect a filter.

Quick quiz

6 questions · tap an answer to check it

1. Which formula gives the correct profit margin for a category?

A SUM([Profit]) / SUM([Sales])

B AVG([Profit] / [Sales])

C [Profit] / [Sales] summed

D MAX([Profit]) / MAX([Sales])

Answer: A. Divide the totals; do not average row ratios.

2. You want each customer's first order date regardless of the view. Which expression?

- **A** {FIXED [Customer ID] : MIN([Order Date])}
- **B** {INCLUDE [Customer ID] : MIN([Order Date])}
- **C** RUNNING_MIN([Order Date])
- **D** MIN([Order Date])

Answer: A. FIXED computes at exactly the customer level.

3. A running total restarts for every category. Why?

- **A** Category is partitioning the calculation (Compute Using is across months)
- **B** The data is wrong
- **C** It is a FIXED LOD
- **D** The filter is in context

Answer: A. Fields not used for addressing partition the calculation.

4. Your FIXED LOD ignores the Region filter. How do you fix it?

- **A** Add the Region filter to context
- **B** Use a measure filter
- **C** Change to a table calculation
- **D** Hide the field

Answer: A. Context filters run before FIXED LODs.

5. **Which table-calculation function returns the previous value in the partition?**

A LOOKUP(SUM([Sales]), -1)

B INDEX()

C SIZE()

D TOTAL(SUM([Sales]))

Answer: A. LOOKUP with offset −1 looks one mark back.

6. **Which expression gives each category's share of its region with Region and Category in the view?**

A SUM([Sales]) / SUM({EXCLUDE [Category] : SUM([Sales])})

B SUM([Sales]) / SUM([Sales])

C {FIXED [Category] : SUM([Sales])}

D RANK(SUM([Sales]))

Answer: A. EXCLUDE removes Category, leaving the region total.

EXERCISE **Practice**

Create: (1) a "Profit status" field that labels each sub-category Loss or Profit and colour a bar chart with it; (2) a 3-month moving average of monthly sales; (3) a cohort table of customers by first-purchase year and order year; (4) a percent-of-region field with EXCLUDE; (5) a Top-N rank of products by profit.

Show solution (worked answer)

(1) `IF SUM([Profit]) < 0 THEN "Loss" ELSE "Profit" END` on Color. (2) `WINDOW_AVG(SUM([Sales]), -2, 0)` with continuous MONTH on Columns. (3) `{FIXED [Customer ID] : MIN([Order Date])}`, YEAR of it on Rows, YEAR(Order Date) on Columns, COUNTD([Customer ID]) on Text. (4) As in the lesson. (5) `RANK(SUM([Profit]))` computed along Product Name, then filter rank ≤ 10 (a table-calculation filter).

INTERVIEW CORNER **Questions you may be asked**

Q: Explain FIXED, INCLUDE and EXCLUDE. FIXED aggregates at exactly the listed dimensions, ignoring the view; INCLUDE adds dimensions to the view's level (finer), then the result is re-aggregated; EXCLUDE removes dimensions from the view's level (coarser). FIXED runs before dimension filters; INCLUDE and EXCLUDE run after them.

Q: Table calculation vs LOD expression? A table calculation runs last, on the aggregated numbers in the view, and depends on layout (Compute Using). An LOD runs in the data source query at a level you define and can be used as a dimension, e.g. for cohorts.

Q: Why might a running total be wrong? Usually Compute Using: it is computing along the wrong field, or a dimension is partitioning it. Check Edit Table Calculation → Specific Dimensions and the sort order.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.